

Introducción a la programación.

M. en C. Saul Villegas.

Universidad Autónoma de la Ciudad de México.

13 de mayo de 2026

Al finalizar este bloque, el estudiante será capaz de:

- Comprender qué es una función.
- Crear funciones propias.
- Diferenciar variables locales y globales.
- Comprender el paso de parámetros.
- Utilizar punteros para modificar variables.
- Aplicar recursividad.
- Separar programas en módulos.

¿Por qué existen las funciones?

Cuando un programa crece

- Repetir código genera errores.
- Cuesta mantenerlo.
- Es difícil entenderlo.

Solución

Usar funciones permite dividir problemas grandes en partes pequeñas, claras y reutilizables.

¿Qué es una función?

- Es un bloque de instrucciones.
- Realiza una tarea específica.
- Puede recibir datos.
- Puede regresar un resultado.

Ejemplo en C

```
int sumar(int a, int b){  
    return a + b;  
}
```

Ventajas de usar funciones

- Reutilización de código.
- Programas más limpios.
- Más fáciles de corregir.
- Facilitan el trabajo en equipo.
- Reducen el tiempo de desarrollo.

Anatomía de una función

Estructura general

```
tipo nombre(parametros){  
    instrucciones;  
    return valor;  
}
```

Ejemplo

```
float promedio(float a, float b){  
    return (a + b) / 2;  
}
```

Partes

- Tipo de retorno.
- Nombre.
- Parámetros.
- Cuerpo.
- Retorno.

Prototipo, definición y llamada

Prototipo

```
int sumar(int a, int b);
```

Definición

```
int sumar(int a, int b){  
    return a + b;  
}
```

Llamada

```
r = sumar(3, 5);
```

Primero se declara, luego se define y finalmente se utiliza.

Paso de parámetros y punteros

1. Paso por valor

```
void cambiar(int x){  
    x = 100;  
}  
int a = 5; cambiar(a);
```

Resultado: a = 5. No cambia.

2. Para modificar la original

```
int a = 5, *p = &a;
```

- a: valor.
- &a: dirección.
- p: guarda dirección.
- *p: accese al contenido.

3. Paso tipo referencia

```
void cambiar(int *x){  
    *x = 100;  
}  
int main(){  
    int a = 5; cambiar(&a);  
}
```

Ahora: a = 100.

Variables locales y globales

Locales

Solo existen dentro de la función.

```
void f(){  
    int x = 5;  
}
```

Globales

Se declaran fuera de las funciones.

```
int contador = 0;
```

Pueden usarse en varias funciones.

Usar preferentemente variables locales

- Reducen efectos secundarios.
- Facilitan pruebas y depuración.
- Hacen más claro el comportamiento de cada función.

Evitar exceso de globales

- Generan errores ocultos.
- Dificultan el mantenimiento.
- Complican el trabajo en equipo.

Visualización del stack

EL STACK EN LLAMADAS A FUNCIONES EN C

[Reiniciar](#)[Tema](#)1) Sin parámetros
sin retorno2) Con parámetros
(valor)3) Con parámetros
(referencia)4) Con retorno
de valor5) Recursividad
(factorial)

CÓDIGO

```
1 #include <stdio.h>
2
3 void saludar() {
4     printf("Hola desde saludar()\n");
5 }
6
7 int main() {
8     printf("Antes de llamar a saludar()\n");
9     saludar(); // llamada a función
10    printf("Después de llamar a saludar()\n");
11    return 0;
12 }
```

Descripción del ejemplo

Ejemplo 1: Sin parámetros y sin retorno.

La función saludar() no recibe parámetros y no devuelve ningún valor.

VISUALIZACIÓN DEL STACK

Paso 2/6

Ejecutando: saludar()



▶ Ejecutar paso a paso

▶▶ Ejecutar hasta el final

◀◀ Retroceder paso

SECUENCIA DE EJECUCIÓN



EJECUCIÓN PARA factorial(4)



En cada llamada recursiva se crea un nuevo marco en el stack. Al llegar al caso base ($n \leq 1$), comienzan los retornos, desduplicando marcos y calculando los resultados.

Idea clave: El stack permite que las funciones se llamen entre sí, se pasen datos y regresen al punto exacto de donde fueron llamadas.

¿CÓMO USAR?

- Selección un ejemplo (pestañas de arriba).
- Presiona "Ejecutar paso a paso" para ver cómo el stack cambia con cada llamada/retorno.
- Observa el stack (pila) a la derecha.
- Lee la explicación de lo que ocurre en cada paso.

EXPLICACIÓN DEL PASO ACTUAL

Paso 2: Llamada a saludar()

Cuando main() llama a saludar():

- Se crea un nuevo marco (stack frame) para saludar().
- Se guarda en ese marco la dirección de retorno (0x0400705), que es la siguiente instrucción en main() después de la llamada.
- El Stack Pointer (SP) se mueve hacia abajo (a direcciones menores).
- La ejecución salta al código de saludar().

COMPONENTES DE UN MARCO (STACK FRAME)



CONCEPTOS CLAVE

El stack es LIFO (Last In, First Out). El último marco que entra es el primero que sale.

El SP (Stack Pointer) siempre apunta al tope del stack.

Cada llamada a función tiene su propio marco independiente.

Al retornar, el marco se elimina y la ejecución continúa en la dirección de retorno guardada.

Caso de estudio: enfriamiento de Newton

Descripción

Se desarrollará un programa en lenguaje C para simular el enfriamiento de un objeto caliente mediante la ley de enfriamiento de Newton.

- Calcula la evolución de la temperatura en pasos discretos.
- Muestra resultados en consola.
- Genera datos para graficarlos posteriormente.

Aplicaciones

Motor, pieza metálica, resistencia eléctrica, horno o una bebida caliente expuesta al ambiente.

Ley de enfriamiento de Newton

$$\frac{dT}{dt} = -k(T - T_a)$$

- $T(t)$: temperatura del objeto.
- T_a : temperatura ambiente.
- k : coeficiente de enfriamiento.
- t : tiempo.

Ecuación discreta

$$T_{n+1} = T_n - k(T_n - T_a)$$

- T_n : temperatura en el paso actual.
- T_{n+1} : temperatura en el siguiente paso.
- T_a : temperatura ambiente.
- k : coeficiente de enfriamiento discreto.
- n : número de iteración.

Objetivo del programa

El programa debe permitir:

- Ingresar temperatura inicial.
- Ingresar temperatura ambiente.
- Ingresar coeficiente de enfriamiento.
- Ingresar número de pasos.
- Calcular la temperatura paso a paso.
- Mostrar una gráfica simple en consola.
- Exportar los datos a un archivo .csv.
- Comparar simulación iterativa y recursiva.

Variables principales

Declaración

```
double T0;  
double Ta;  
double k;  
int pasos;
```

Significado

- T0: temperatura inicial del objeto.
- Ta: temperatura ambiente.
- k: coeficiente de enfriamiento.
- pasos: número de iteraciones.

Función: siguiente temperatura

Prototipo

```
double calcularSiguienteTemperatura(double T,  
                                   double Ta,  
                                   double k);
```

- Recibe la temperatura actual, la ambiente y el coeficiente.
- Devuelve la temperatura del siguiente paso.

Ecuación usada

$$T_{n+1} = T_n - k(T_n - T_a)$$

Actualización por referencia

Prototipo

```
void actualizarTemperatura(double *T,  
                           double Ta,  
                           double k);
```

- Recibe la dirección de memoria de la temperatura.
- Modifica directamente su valor.
- Permite explicar punteros y paso por referencia.

Actualización interna

$$T \leftarrow T - k(T - T_a)$$

Mostrar estado y gráfica en consola

Mostrar estado

```
void mostrarEstado(int paso, double T);
```

Ejemplo:

```
Paso 5 | Temperatura: 67.42 C
```

Gráfica en consola

```
void graficarTemperatura(double T);
```

```
Paso 0 | 90.00 C | #####
```

```
Paso 1 | 84.80 C | #####
```

```
Paso 2 | 80.02 C | #####
```

Prototipo

```
double simularRecursivo(double T,  
                        double Ta,  
                        double k,  
                        int pasos);
```

- Caso base: si $\text{pasos} == 0$, la temperatura final es T .
- Caso recursivo: calcula el siguiente valor y llama otra vez.

Llamada recursiva en código

```
return simularRecursivo(  
    calcularSiguienteTemperatura(T, Ta, k),  
    Ta,  
    k,  
    pasos - 1  
);
```

Idea clave

Cada llamada crea un nuevo frame en el stack hasta llegar al caso base. Después, las llamadas regresan en orden inverso.

Flujo general del programa

Inicio

Pedir T0

Pedir Ta

Pedir k

Pedir pasos

Mostrar menu

1. Simulacion paso a paso

2. Simulacion recursiva

3. Exportar datos CSV

4. Salir

Fin

Ejemplo numérico

Datos iniciales

$$T_0 = 90\text{ °C}, \quad T_a = 25\text{ °C}, \quad k = 0,08, \quad \text{pasos} = 20$$

Primera iteración

$$T_1 = 90 - 0,08(90 - 25) = 84,8$$

Segunda iteración

$$T_2 = 84,8 - 0,08(84,8 - 25) = 80,016$$

Qué debe observarse en la gráfica

- La temperatura disminuye rápido al inicio porque $T_n - T_a$ es grande.
- Al acercarse a la temperatura ambiente, el enfriamiento se vuelve más lento.
- El sistema tiende a la temperatura ambiente.

Límite esperado

$$\lim_{n \rightarrow \infty} T_n = T_a$$

Conceptos de programación involucrados

- Variables
- Condicionales
- Ciclos
- Funciones con retorno
- Funciones void
- Paso por valor
- Paso por referencia
- Punteros
- Recursividad
- Stack y archivos CSV

Paso por valor

```
calcularSiguieteTemperatura(T, Ta, k);
```

Se crea un frame con copias de T, Ta y k.

Paso por referencia

```
actualizarTemperatura(&T, Ta, k);
```

Se crea un frame con la dirección de T, por eso la función puede modificar la variable original.

Resultado esperado

Salida en consola

```
Paso 0 | 90.00 C | #####
Paso 1 | 84.80 C | #####
Paso 2 | 80.02 C | #####
Paso 3 | 75.62 C | #####
...
Paso 20 | 37.27 C | #####
```

Archivo CSV

```
datos_enfriamiento.csv
Paso, Temperatura
0, 90.00
1, 84.80
2, 80.02
3, 75.62
```

Ejemplo: factorial

```
int fact(int n){  
    if(n <= 1) return 1;  
    return n * fact(n - 1);  
}
```

Idea clave

Una función puede llamarse a sí misma para resolver un problema en pasos más pequeños.

Ejemplo de evaluación

$$\text{fact}(4) = 4 * 3 * 2 * 1$$

¿Cuándo usar recursividad?

La recursividad es útil en problemas como:

- Factorial.
- Fibonacci.
- Árboles.
- Búsquedas.
- Algoritmos matemáticos.

Criterio práctico

Se recomienda cuando un problema se descompone naturalmente en subproblemas similares.

Separar el programa en archivos

- `main.c`
- `operaciones.c`
- `operaciones.h`

Ventajas

- Orden.
- Mantenimiento.
- Escalabilidad.
- Trabajo colaborativo.

Ejemplo modular

operaciones.h

```
int sumar(int, int);
```

operaciones.c

```
int sumar(int a, int b){  
    return a + b;  
}
```

main.c

```
printf("%d", sumar(2, 3));
```

Situaciones que debemos evitar

- No declarar el prototipo.
- Usar tipos incompatibles.
- No retornar un valor cuando corresponde.
- Confundir *p con p.
- Intentar usar variables fuera de su alcance.

Crear un programa que:

- 1 Lea dos números.
- 2 Use una función de suma.
- 3 Use una función de multiplicación.
- 4 Use un puntero para intercambiar valores.
- 5 Muestre los resultados.

Meta

Integrar funciones, parámetros y punteros en un mismo ejercicio.

Las funciones permiten escribir programas profesionales

- Funciones.
- Parámetros.
- Punteros.
- Variables locales y globales.
- Recursividad.
- Modularidad.

Reflexiona y responde

¿Por qué un programa dividido en funciones es mejor que uno de 500 líneas seguidas?

Modularidad = claridad + reutilización + mantenimiento

Que el estudiante identifique, ejemplifique, aplique y codifique soluciones en lenguaje C utilizando:

- Arreglos unidimensionales.
- Arreglos bidimensionales.
- Estructuras.
- Solución de problemas con vectores y matrices.

¿Por qué necesitamos arreglos?

Supongamos que queremos almacenar 100 calificaciones.

Sin arreglos:

```
int cal1;  
int cal2;  
int cal3;  
...
```

1

Con arreglos:

```
int calificaciones[100];
```

Concepto de Arreglo

Un arreglo es una colección de elementos del mismo tipo almacenados bajo un mismo nombre.

Sintaxis general

```
tipo nombre[tamano];
```

Ejemplos

```
int edades[5];  
float temperaturas[30];  
char nombre[20];
```

Índices en un Arreglo

En C, los arreglos comienzan en el índice 0.

Índice	0	1	2	3	4
Valor	?	?	?	?	?

```
int numeros[5];
```

Inicialización de Arreglos

```
int numeros[5] = {10, 20, 30, 40, 50};
```

Acceso a elementos

```
printf("%d", numeros[0]);  
printf("%d", numeros[3]);
```

Resultado

```
10  
40
```


Inicialización Parcial

Si no se inicializan todos los elementos, los restantes toman valor cero.

```
int a[4] = {2, 7};
```

Índice	Valor
0	2
1	7
2	0
3	0

Captura de Datos en un Arreglo

```
#include <stdio.h>

int main() {
    int numeros[5];
    int i;

    for(i = 0; i < 5; i++) {
        printf("Numero %d: ", i);
        scanf(" %d", &numeros[i]);
    }

    return 0;
}
```

Recorrido de un Arreglo

```
#include <stdio.h>

int main() {
    int numeros[5] = {4, 8, 15, 16, 23};
    int i;

    for(i = 0; i < 5; i++) {
        printf("%d\n", numeros[i]);
    }

    return 0;
}
```

Error Común: Índice Fuera de Rango

```
int a[2] = {3, 6};  
printf("%d", a[7]);
```

Esto puede provocar:

- Lectura de memoria no reservada.
- Valores basura.
- Comportamiento impredecible.
- Fallos de ejecución.

Guard Clauses para Índices

Una forma segura de acceder a un arreglo es validar antes el índice.

```
if(indice < 0 || indice >= 5) {  
    printf("Índice invalido");  
    return 1;  
}  
  
printf("%d", numeros[indice]);
```

Idea clave: nunca confiar completamente en la entrada del usuario.

Ejercicio 1: Promedio de Calificaciones

Crear un programa que:

- Capture 10 calificaciones.
- Calcule el promedio.
- Encuentre la calificación mayor.
- Encuentre la calificación menor.

Conceptos utilizados

Arreglos, ciclos y condicionales.

Arreglos Bidimensionales

Un arreglo bidimensional permite representar datos en forma de tabla o matriz.

Sintaxis

```
tipo nombre[filas][columnas];
```

Ejemplo

```
int matriz[3][3];
```

Representación de una Matriz

```
int matriz[2][3];
```

	C0	C1	C2
F0	?	?	?
F1	?	?	?

Inicialización de una Matriz

```
int matriz[2][2] = {  
    {1, 2},  
    {3, 4}  
};
```

1	2
3	4

Recorrido de una Matriz

```
#include <stdio.h>

int main() {
    int matriz[2][2] = {{1, 2}, {3, 4}};
    int i, j;

    for(i = 0; i < 2; i++) {
        for(j = 0; j < 2; j++) {
            printf("%d ", matriz[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Las matrices se utilizan en:

- Imágenes digitales.
- Tableros de videojuegos.
- Modelos matemáticos.
- Sistemas físicos.
- Procesamiento de datos.
- Inteligencia artificial.

¿Por qué necesitamos estructuras?

Un estudiante puede tener:

- Nombre.
- Edad.
- Promedio.

Pero estos datos no son del mismo tipo.

Solución

Usar una estructura.

Concepto de struct

Una estructura permite agrupar variables relacionadas bajo un mismo tipo de dato.

```
struct Alumno {  
    char nombre[30];  
    int edad;  
    float promedio;  
};
```

Declaración de Variables Estructura

```
struct Alumno alumno1;
```

Asignación de valores

```
alumno1.edad = 20;  
alumno1.promedio = 9.5;
```

Captura de Datos en una Estructura

```
#include <stdio.h>

struct Alumno {
    char nombre[30];
    int edad;
    float promedio;
};

int main() {
    struct Alumno a;

    printf("Nombre: ");
    scanf("%s", a.nombre);

    printf("Edad: ");
    scanf("%d", &a.edad);

    printf("Promedio: ");
    scanf("%f", &a.promedio);

    return 0;
}
```

Mostrar Información de una Estructura

```
printf("Nombre: %s\n", a.nombre);  
printf("Edad: %d\n", a.edad);  
printf("Promedio: %.2f\n", a.promedio);
```

El operador punto `.` permite acceder a los campos de la estructura.

Podemos almacenar muchos registros usando un arreglo de estructuras.

```
struct Alumno grupo[30];
```

Aplicaciones

Registros escolares, inventarios, catálogos y bases de datos simples.

Sistema de Control Escolar

Crear un programa que:

- Guarde 5 alumnos.
- Capture nombre, edad y promedio.
- Calcule el promedio grupal.
- Muestre el alumno con mejor promedio.
- Muestre el alumno con mayor edad.

En arreglos

- Salirse del rango del índice.
- Confundir tamaño con último índice.
- No inicializar elementos.

En estructuras

- Olvidar el punto y coma al cerrar el struct.
- Usar mal el operador punto.
- Mezclar tipos incompatibles.

Arreglos, matrices y estructuras aparecen en:

- Sistemas embebidos.
- Lectura de sensores.
- Robótica.
- Control automático.
- Procesamiento de señales.
- Sistemas de monitoreo.

Diseñar un programa que administre información de sensores.

Debe incluir:

- Un arreglo para almacenar mediciones.
- Una matriz para organizar datos por sensor y tiempo.
- Una estructura para guardar información de cada sensor.

Ideas clave:

- Los arreglos almacenan datos del mismo tipo.
- Las matrices representan información bidimensional.
- Las estructuras agrupan datos de distintos tipos.
- Estos conceptos son base para programación avanzada.

¿Dudas?

¿Qué diferencia existe entre arreglo, matriz y estructura?